

1. Monthly Application Volume and Approval Distribution

SQL (conceptual):

```
SELECT DATE_TRUNC('month', a.submittedTime) AS month,
       COUNT(*) AS total_applications,
       SUM(CASE WHEN ar.result = 'approved' THEN 1 ELSE 0 END) AS approved,
       SUM(CASE WHEN ar.result = 'rejected' THEN 1 ELSE 0 END) AS rejected,
       SUM(CASE WHEN ar.result = 'pending' THEN 1 ELSE 0 END) AS pending
FROM application a
LEFT JOIN application_review ar ON a.applicationId = ar.applicationId
LEFT JOIN application_result r ON ar.resultId = r.resultId
GROUP BY month
ORDER BY month;
```

Visualization: Line chart (monthly applications) + stacked bar chart (approved / rejected / pending)

Practical Use: Helps administrators monitor system demand and approval efficiency over time

2. Experiments with the Highest Number of Applications (Top 10)

SQL (conceptual):

```
SELECT e.experimentId, e.title, COUNT(a.applicationId) AS application_count
FROM application a
LEFT JOIN experiment e ON a.experimentId = e.experimentId
GROUP BY e.experimentId, e.title
ORDER BY application_count DESC
LIMIT 10;
```

Visualization: Ranked table (Top 10 experiments)

Practical Use: Identifies popular research directions and high-demand experiments

3. Most Requested Facilities and Equipment

SQL (conceptual):

```
SELECT f.facilityId, f.name, COUNT(a.applicationId) AS request_count
FROM application a
LEFT JOIN facility f ON a.facilityId = f.facilityId
```

```
GROUP BY f.facilityId, f.name  
ORDER BY request_count DESC;
```

-- similar query for equipment

Visualization: Table (ranked by request count)

Practical Use: Helps allocate resources and prioritize expansion of high-demand facilities/equipment

4. Ratio of Actual Usage Time to Allocated Time

SQL (conceptual):

```
SELECT ua.allocationId,  
       SUM(ur.duration) / SUM(ua.allocatedDuration) AS usage_ratio  
FROM usage_record ur  
JOIN actual_allocation ua ON ur.allocationId = ua.allocationId  
GROUP BY ua.allocationId;
```

Visualization: Pie chart (used vs unused allocation time)

Practical Use: Evaluates resource utilization efficiency and identifies waste

5. Equipment with the Most and Least Maintenance Activities

SQL (conceptual):

```
SELECT e.equipmentId, COUNT(em.activityId) AS maintenance_count  
FROM equipment e  
LEFT JOIN equipment_maintenance em ON e.equipmentId = em.equipmentId  
GROUP BY e.equipmentId  
ORDER BY maintenance_count DESC;
```

Visualization: Table (Top 3 highest and Top 3 lowest)

Practical Use: Identifies unreliable equipment and those that may be under-maintained

6. Facilities with Equipment Requiring the Most Maintenance

SQL (conceptual):

```
SELECT f.location, COUNT(em.activityId) AS total_maintenance
FROM equipment_maintenance em
JOIN equipment e ON em.equipmentId = e.equipmentId
JOIN facility f ON e.facilityId = f.facilityId
GROUP BY f.location
ORDER BY total_maintenance DESC;
```

Visualization: Table

Practical Use: Helps allocate more technical staff to high-maintenance locations

7. Equipment with Most Unscheduled Maintenance and Responsible Users

SQL (conceptual):

```
SELECT e.equipmentId, p.personId, COUNT(em.activityId) AS unscheduled_count
FROM equipment_maintenance em
JOIN equipment e ON em.equipmentId = e.equipmentId
JOIN usage_record ur ON e.equipmentId = ur.equipmentId
JOIN credential c ON ur.credentialId = c.credentialId
JOIN person p ON c.personId = p.personId
WHERE em.isScheduled = 'No'
GROUP BY e.equipmentId, p.personId
ORDER BY unscheduled_count DESC;
```

Visualization: Table

Practical Use: Helps detect improper equipment usage and identify responsible users

8. Technical Staff Maintenance Workload Ranking

SQL (conceptual):

```
SELECT ts.personId, COUNT(em.activityId) AS maintenance_tasks
FROM technical_staff ts
LEFT JOIN equipment_maintenance em ON ts.personId = em.personId
GROUP BY ts.personId
ORDER BY maintenance_tasks DESC;
```

Visualization: Table (ranked staff workload)

Practical Use: Supports workforce planning and task redistribution

9. Real-Time Status of All Facilities and Equipment

SQL (conceptual):

```
SELECT f.facilityId, f.name, f.status,  
       e.equipmentId, e.status  
FROM facility f  
LEFT JOIN equipment e ON f.facilityId = e.facilityId;
```

Visualization: Table (current status overview)

Practical Use: Provides operational visibility for monitoring availability

10. Unpaid Charges and Responsible Parties

SQL (conceptual):

```
SELECT c.chargeId, fs.sourceName, p.personId  
FROM charge c  
LEFT JOIN payment pmt ON c.chargeId = pmt.chargeId  
JOIN charge_allocation ca ON c.chargeId = ca.chargeId  
JOIN funding_source fs ON ca.fundingSourceId = fs.fundingSourceId  
JOIN person p ON fs.personId = p.personId  
WHERE pmt.paymentId IS NULL;
```

Visualization: Pie chart (paid vs unpaid) + table (responsible persons)

Practical Use: Identifies financial risk and tracks accountability

11. Institutions with the Highest Total Charges

SQL (conceptual):

```
SELECT i.institutionId, i.name, SUM(c.amount) AS total_charge  
FROM charge c  
JOIN person p ON c.personId = p.personId  
JOIN institution i ON p.institutionId = i.institutionId  
GROUP BY i.institutionId, i.name  
ORDER BY total_charge DESC;
```

Visualization: Table (ranked institutions)

Practical Use: Helps understand revenue sources and institutional resource consumption

